

NetSage AI

An AI-Assisted Network Troubleshooting Assistant with Human-in-the-Loop Review

Student Name	Tarun Pandey
Enrollment Number	25SCS1003004006
Program & Semester	B.Tech CSE — 3RD Semester
Organization	Cisco Systems, Inc.
Batch	2026-27



IILM UNIVERSITY

Greater Noida, Uttar Pradesh

School of Computer Science
and Engineering

Agenda

01 The Problem

Why symptom-to-root-cause is hard

02 Problem Statement

The Cisco requirement

03 Objectives & Scope

What the system must do

04 System Architecture

How the layers fit together

05 Case Dataset

30 Packet Tracer scenarios

06 Rule Checker

Deterministic validation

07 AI Engine & Prompts

Structured, evidence-bound output

08 Human-in-the-Loop

Accept / Edit / Reject

09 Responsible AI

When the AI was wrong

10 Results & Demo

Verification and walkthrough

THE PROBLEM

One symptom, six possible root causes

“A PC receives an IP address but cannot reach the server.”

A junior engineer can run the commands — but cannot tell which of these is actually responsible:

VLAN

Routing

DHCP

DNS

ACL

NAT

Each cause requires a different diagnostic path. Choosing the wrong one costs time that is critical during an outage — and an AI that answers confidently but incorrectly makes this worse, not better.

6+

Fault domains for one symptom

30

Lab cases built to cover them

100%

Diagnoses requiring human review

Build an AI troubleshooting helper with human review

Read symptoms and show-command output, suggest likely causes and next steps, and ALWAYS require a human to review before accepting the fix.

● Symptom-to-cause mapping

One symptom maps to many faults; the system must reason across domains rather than pattern-match to the most common answer.

● Evidence grounding

A language model will invent plausible command output. Every claim must cite supplied evidence, and gaps must be named explicitly.

● Overconfidence

Models express uncertainty poorly. Confidence must be tied to evidence strength and lowered when evidence is thin or contradictory.

● Oversight

If a wrong diagnosis can be auto-accepted, risk scales with use. Review must be structurally impossible to bypass.

Project Objectives

1

Build a realistic case dataset

30+ Packet Tracer cases across 8 fault categories with authentic show-command evidence.

3

Implement a deterministic checker

Pure Python, zero AI — six reproducible configuration checks.

5

Enforce human review in code

Accept / Edit / Reject by a named reviewer before any diagnosis is final.

2

Design a structured prompt library

Constrain the AI to supplied evidence; return a fixed JSON schema with worked examples.

4

Make the AI layer modular

Live LLM via environment variable, with a labelled fallback so no API key is required.

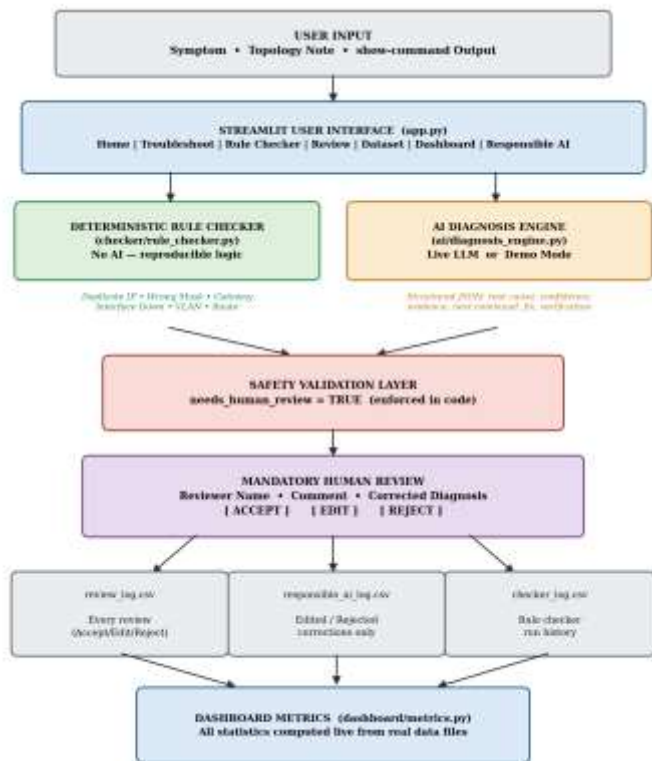
6

Document where the AI was wrong

A Responsible AI log of corrections, with the reason and the lesson for each.

HOW IT FITS TOGETHER

System Architecture



● Presentation

Streamlit, 8 pages. Captures input and renders output only — no diagnostic logic.

● Deterministic checker

Pure Python, no AI. Reproducible, explainable results fed to the AI as extra evidence.

● AI diagnosis

Loads the prompt from a markdown file at runtime; live LLM or labelled Demo Mode.

● Safety validation

`needs_human_review` is overwritten to `TRUE` in code — a model response cannot bypass it.

● Human review

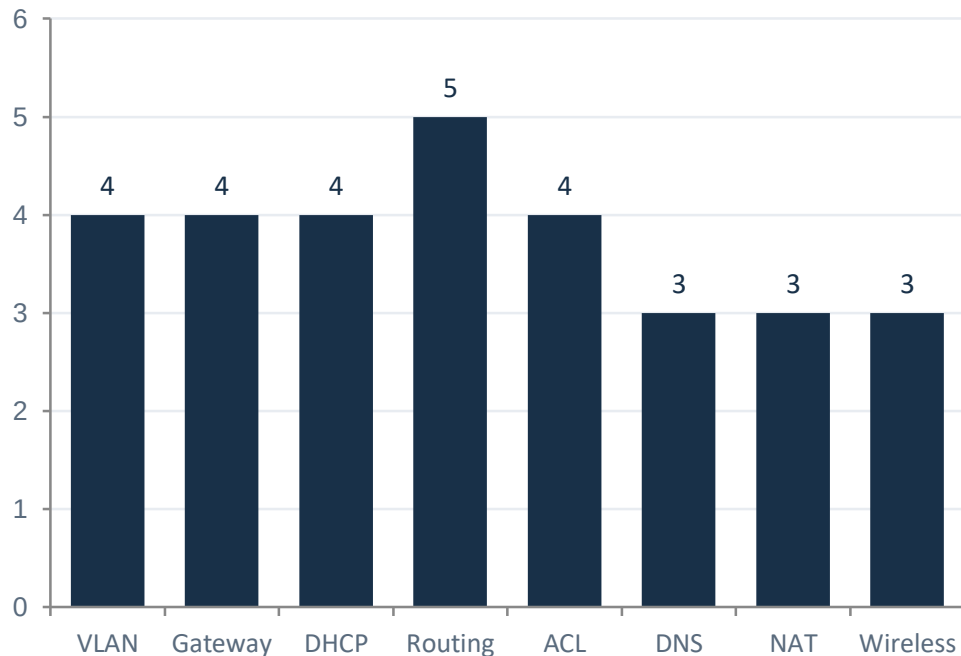
Named reviewer must Accept, Edit or Reject. Edits require a correction.

● Persistence & metrics

Three CSV logs; every dashboard statistic is computed from them, never hard-coded.

Case Dataset — 30 Packet Tracer Scenarios

Cases by fault category



Every case carries 13 fields

- Symptom — what the user observed
- Topology note — relevant devices
- show outputs — authentic Cisco IOS
- Expected fault — verified answer
- OSI layer + concept tag
- Severity rating
- Expected next command
- Expected fix + verification command

Only real Cisco commands are used — show ip route, show vlan brief, show access-lists, show ip nat translations, nslookup. No invented syntax.

NO AI — FULLY REPRODUCIBLE

Deterministic Rule Checker

Pure Python logic that catches mechanical configuration errors before the AI is even called. Every result is explainable line by line.

Duplicate IP

Same address on two devices

Wrong subnet mask

Four principled sub-checks

Gateway mismatch

Host points at wrong gateway

Interface down

Administratively disabled port

Missing VLAN

Referenced but never created

Missing route

No valid next hop

A defect found and fixed during development

The mask check originally compared each device against the most common mask in the input — which flagged a legitimate /30 point-to-point WAN link as an error. It was rebuilt around four principled sub-checks, and regression tests now prevent that false positive from returning.

AI Diagnosis Engine

The prompt is loaded from a markdown file at runtime, so the AI's behaviour can be changed without touching program code. Output is always a fixed JSON schema.

```
{
  "case_id":      "C021",
  "root_cause":   "ACL 101 denies ...",
  "confidence":   0.70,
  "confidence_label": "Medium",
  "osi_layer":    "Layer 3 / Layer 4",
  "issue_type":   "ACL",
  "evidence":     [ "show access-lists..." ],
  "reasoning_summary": "...",
  "next_command": "show access-lists",
  "fix_steps":    [ "..." ],
  "verification_steps": [ "..." ],
  "needs_human_review": true
}
```

- **Cite real evidence**

Every claim points to a specific supplied show-output line.

- **Never fabricate output**

The model may not invent commands, interfaces or addresses.

- **Calibrate confidence**

Lowered when evidence is thin, ambiguous or contradictory.

- **Name what is missing**

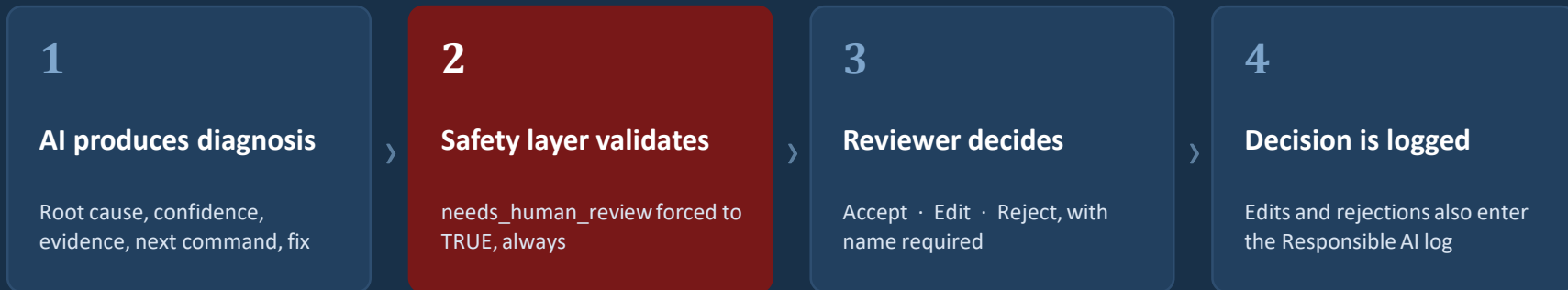
Recommends the next command that would close the gap.

- **Never claim it is fixed**

It recommends; a human applies and verifies.

The AI never has the final say

Human review is enforced in program code — not merely requested in the prompt. A malformed or adversarial model response cannot bypass it.



Every diagnosis — live or demo — requires an explicit decision from a named reviewer before it is considered final. Edit and Reject cannot be submitted without a corrected diagnosis.

Responsible AI — When the AI Was Wrong

Six documented cases where the initial AI diagnosis was incorrect, incomplete or overconfident. The log grows automatically whenever a reviewer edits or rejects a diagnosis.

Case C021 — logged as RAI-001

AI said “DNS is the likely cause.” Confidence 0.55

Actually ACL 101 explicitly denies the VLAN 30 source subnet

Decision Edited by reviewer

Why it erred It prioritised DNS despite show access-lists containing an explicit deny rule matching the affected subnet

Fabricated a cause

Blamed cabling with no supporting evidence in the input

Blamed an outsider

Claimed an ISP outage when internal config explained it

Was overconfident

Assigned 0.85 to a hypothesis the evidence contradicted

These are not hidden failures — they are the argument for why human oversight is required.

EVERYTHING MEASURED, NOTHING ASSUMED

Results & Verification

30

Cases across 8 fault categories

28

Automated unit tests passing

22

Requirement checks passing

6

Documented AI corrections

One command verifies the whole project

```
python verify_project.py
```

Maps every requirement of the problem statement to an executable check and reports pass or fail for each.

● All 30 cases diagnose

An earlier build degraded on 23 of 30 — found in review and fixed

● All 6 checks demonstrated

The sample input seeds one fault of every category

● Zero auto-accepted diagnoses

Verified across every code path, including malformed input

WHAT I WILL SHOW

Live Demonstration

1

Select a broken case

Case C001 — a PC in VLAN 30 cannot reach a VLAN 10 server

2

Show the evidence

Authentic show vlan brief, show interfaces trunk, show ip route output

3

Run the rule checker

Deterministic findings surface before the AI is called

4

Run the AI diagnosis

Root cause, confidence, cited evidence, next command, fix, verification

5

Review as a human

Edit the diagnosis and supply a correction

6

Show the logs update

Responsible AI log and dashboard statistics change live

Runs entirely offline in Demo Mode — no API key, no network dependency, nothing that can fail mid-demonstration.

Limitations & Future Scope

Current limitations

- The checker parses a simplified inventory format, not raw show running-config output.
- In Demo Mode, cases outside the seven curated responses are dataset lookups — labelled as such and capped at Medium confidence.
- A single LLM provider is integrated; others need an equivalent adapter.
- CSV-backed and single-user — appropriate for a student project, not concurrent production use.

Future scope

- Parse raw Cisco show output directly with structured parsers.
- Multi-provider LLM support with automatic failover between models.
- Multi-reviewer accounts with per-reviewer calibration statistics over time.
- Expand the dataset with real lab failures submitted by students and instructors.

CONCLUSION

Accelerate the diagnosis. Keep the engineer accountable.

NetSage AI shows that an AI assistant can add real value to fault diagnosis without displacing human judgement — by grounding every claim in supplied evidence, enforcing review in code rather than in a prompt, and keeping an open record of its own mistakes.

Evidence-bound

Every claim cites supplied output

Structurally safe

Review enforced in code

Openly fallible

Mistakes logged, not hidden